

A PROJECT REPORT ON

PHISHING WEBSITE AND MALICIOUS URL DETECTION BROWSER EXTENSION

Domain: Cyber Security / Machine Learning

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	22
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

Phishing is a cyber attack in which criminals create fake websites that imitate banks, payment apps, delivery companies, government portals or online shops in order to steal passwords, OTPs, card numbers and personal information. Links to these websites are sent through SMS, WhatsApp, e-mail and social media with messages such as 'Your KYC is expiring', 'Your parcel is on hold' or 'You have won a prize'.

Phishing links are the most common route for credential theft. The Anti-Phishing Working Group reports hundreds of thousands of unique phishing sites every quarter. Blacklists maintained by browsers and security companies help, but new phishing sites appear every minute and many are active for only a few hours, so blacklists lag behind. Ordinary users, especially students and elderly people, cannot judge the safety of a URL by looking at it, because attackers use look-alike domains, URL shorteners and HTTPS certificates.

This project, PhishGuard, is a browser extension that checks every visited link in real time and warns the user when a site looks like phishing. It analyses lexical features of the URL, domain age, SSL certificate information and page content features (such as password forms posting to other domains and hidden iframes) using a trained Random Forest / XGBoost classifier. Risky pages are blocked with a warning, users can report false results, and the model is updated periodically.

LIMITATIONS

1. Phishing links are the most common route for credential theft.
2. Blacklists lag behind newly created scam sites.
3. Users cannot judge URL safety by eye.
4. HTTPS padlock gives a false sense of security.

1.1 History of Domain

The term 'phishing' was first used in 1996 in attacks on America Online users. Early defences were blacklists such as PhishTank (2006) and Google Safe Browsing (2007), which are integrated into browsers and protect billions of users by checking URLs against lists of known malicious sites.

Because blacklists cannot detect new sites, researchers developed heuristic and machine learning approaches. CANTINA (2007) used TF-IDF of page content, and later work by Ma et al. (2009) showed that lexical and host-based URL features with online learning can identify malicious URLs accurately. The UCI Phishing Websites dataset (2012) and later datasets enabled widespread research using Random Forests, gradient boosting and deep learning.

1.2 Overview of the Project

When the user navigates to a URL, the extension's background service worker intercepts it through the webNavigation API and first checks a local cache and an allow-list of popular domains. For unknown URLs it extracts lexical features locally (length, number of dots, hyphens, '@' symbol, IP address use, suspicious keywords, entropy, TLD) and sends the URL to the PhishGuard API.

The API fetches WHOIS domain age and SSL certificate details, and the content script reports page features such as the presence of password fields, forms that post to external domains, hidden iframes, the ratio of external links and brand names in the title that do not match the domain. An XGBoost classifier produces a phishing probability. If the probability exceeds the threshold, the page is blocked with a full-screen warning explaining the reasons, with an option to go back or proceed. Users can report false positives or missed phishing pages, and reviewed reports are used to retrain the model.

1.3 Motivation

One click on a fake KYC or delivery link can empty a bank account, and the people most targeted are those least able to recognise the trick.

- **Motivation:** To protect users at the moment of risk by checking every link automatically and explaining the danger.
- **Outcome:** Fewer successful phishing attacks, increased awareness, and fast detection of new scam sites.

1.4 Aim of the Project

The project is entitled as “**PHISHING WEBSITE AND MALICIOUS URL DETECTION BROWSER EXTENSION**”. The aim of this project is to develop a Chrome browser extension backed by a machine learning API that detects phishing and malicious URLs in real time using URL, domain and page content features, warns or blocks risky pages, and improves through user feedback.

1.5 Objectives

- To extract lexical, domain and content features of web pages.
- To train and compare Random Forest and XGBoost classifiers on phishing datasets.
- To intercept navigation in the browser and score URLs in real time.
- To display explanatory warnings and block risky pages.
- To collect user reports and retrain the model periodically.

1.6 Literature Review

Zhang, Hong and Cranor (2007): Proposed CANTINA, a content-based approach that uses TF-IDF to extract key terms from a page and searches them to check whether the domain appears in the top results, detecting about 90% of phishing sites.

Ma, Saul, Savage and Voelker (2009): Showed that lexical and host-based features of URLs with statistical learning can classify malicious URLs with 95-99% accuracy without examining page content.

Mohammad, Thabtah and McCluskey (2012): Identified important features for phishing website prediction and contributed the UCI Phishing Websites dataset with 30

features including URL, domain and HTML characteristics.

Sahingo et al. (2019): Applied machine learning on NLP-based URL features and found Random Forest to give 97.98% accuracy on a large dataset of legitimate and phishing URLs.

Anti-Phishing Working Group: APWG Phishing Activity Trends Reports document the large and growing number of phishing sites and the increasing use of HTTPS by phishing sites.

PhishTank (OpenDNS/Cisco): A community-driven database of verified phishing URLs used as a source of positive examples for training and evaluation.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

Currently users rely on browser built-in blacklists (Google Safe Browsing), antivirus software and their own judgement. Blacklists protect against known sites but not newly created ones, and many antivirus products do not inspect URLs in the browser in real time.

DISADVANTAGES

- New phishing sites are not detected until they are reported.
- Users get no explanation of why a site is risky.
- HTTPS phishing sites look trustworthy.
- Look-alike domains fool users.
- No easy way for users to report sites.

2.2 Proposed System

The proposed PhishGuard extension detects phishing using a machine learning model on URL, domain and page features, which works even for brand-new sites. It warns users with clear reasons, blocks highly risky pages, and allows reporting, with periodic retraining to adapt to new attack patterns.

ADVANTAGES

- Detects zero-hour phishing sites not yet on blacklists.
- Real-time protection on every navigation.
- Explains the warning in simple language.
- Lightweight with local caching.
- Continuously improved with user feedback.

2.3 Project Schedule

The development of PhishGuard follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the PhishGuard system are listed below.

- **URL interception:** Capture URLs on navigation.
- **Lexical features:** Compute URL length, dots, hyphens, @, IP use, entropy, keywords.
- **Domain features:** Fetch domain age and SSL issuer/validity.
- **Content features:** Analyse forms, password fields, iframes, external links.
- **Classification:** Predict phishing probability.
- **Warning and blocking:** Show warning page with reasons for risky URLs.
- **User reports:** Report false positive / phishing.
- **Model update:** Periodic retraining and versioned deployment.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Latency:** Verdict within 500 ms for most URLs.
- **Accuracy:** F1-score above 0.95 on test data.

- **Privacy:** Only the URL and computed features are sent, not page content or personal data.
- **Performance:** Minimal impact on browsing speed; caching of results.
- **Usability:** Clear warning page in simple language.
- **Reliability:** Fallback to lexical-only local model when offline.

3.3 User Requirements

- Automatic protection without extra effort
- Clear warning with reasons
- Option to go back to safety
- Report wrong decisions
- See history of blocked sites

3.4 System Requirements

- Chrome browser (Manifest V3)
- FastAPI server with trained model
- WHOIS and SSL lookup
- Database for reports and cache
- PhishTank / public datasets for training

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i3 Processor 2.4 GHz or above
Hard Disk : 256 GB
RAM : 8 GB
Monitor : 15" colour monitor

4.2 Software Specification

Operating System : Windows 10 / Ubuntu 22.04
Extension : JavaScript, Chrome Extension API (Manifest V3)
Back End : Python 3.10, FastAPI
ML Libraries : scikit-learn, XGBoost
Lookup Libraries : python-whois, ssl, tldextract
Database : SQLite / PostgreSQL
Dataset : PhishTank, UCI Phishing Websites, Tranco top sites (legitimate)

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed PhishGuard system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Chrome's extension APIs allow observing navigation and injecting content scripts. Python ML libraries and WHOIS/SSL lookups are freely available. The project is technically feasible.

ECONOMIC FEASIBILITY

The extension is free to distribute, and the API runs on a small server. Preventing a single fraud saves users far more than the system costs. It is economically feasible.

OPERATIONAL FEASIBILITY

Users only install the extension once; protection is automatic. The warning page is simple to understand. It is operationally feasible.

5.2 About the Software

CHROME EXTENSION API (MANIFEST V3)

Chrome extensions are small software programs that customise the browsing experience. They are built with HTML, CSS and JavaScript and use the chrome.* APIs such as webNavigation, tabs, storage and declarativeNetRequest.

Manifest V3 uses a background service worker that can observe every navigation and communicate with a content script injected into the page to show warnings.

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

SCIKIT-LEARN

Scikit-learn is a free machine learning library for Python built on NumPy, SciPy and Matplotlib. It provides simple and efficient tools for predictive data analysis including classification, regression, clustering and dimensionality reduction.

It contains implementations of popular algorithms such as Random Forest, Support Vector Machine, Logistic Regression, k-Nearest Neighbours and Isolation Forest, along with utilities for train-test splitting, cross-validation, feature scaling, pipelines and evaluation metrics such as accuracy, precision, recall and F1-score.

- Consistent fit / predict API for all models
- Built-in preprocessing and model selection tools
- Well documented and production tested

XGBOOST

XGBoost (Extreme Gradient Boosting) is an optimised, distributed gradient boosting library designed to be highly efficient, flexible and portable. It implements machine learning algorithms under the gradient boosting framework, where many shallow decision trees are added one after another and each new tree corrects the errors of the previous ones.

XGBoost handles missing values automatically, supports regularisation to avoid over-fitting, and allows class weighting through the `scale_pos_weight` parameter, which makes it very suitable for imbalanced problems such as fraud and fault detection on tabular data.

- High accuracy on tabular data
- Fast training with parallel tree construction
- Feature importance for explainability

FASTAPI

FastAPI is a modern, high-performance web framework for building APIs with Python based on standard Python type hints. It is built on Starlette for the web parts and Pydantic for data validation.

FastAPI automatically generates interactive API documentation (Swagger UI and ReDoc), validates request bodies, and supports asynchronous request handling, which gives performance comparable to Node.js and Go. It is well suited for serving machine learning models with low latency.

- Automatic request validation with Pydantic
- Auto-generated OpenAPI documentation
- Asynchronous and very fast

POSTGRESQL

PostgreSQL is a powerful, open-source object-relational database system with over 35 years of active development. It has earned a strong reputation for reliability, feature

robustness and performance.

PostgreSQL supports ACID transactions, foreign keys, views, triggers, stored procedures and JSON data types. Its extension system allows additional capabilities such as PostGIS for geographic data and pgvector for similarity search.

- Fully ACID compliant
- Advanced SQL and JSON support
- Extensible with PostGIS and other extensions

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of PhishGuard is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

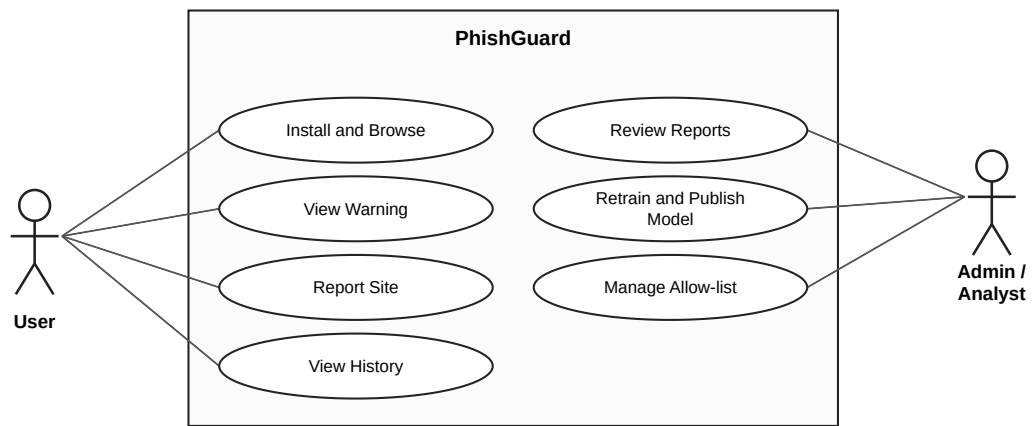


Fig 1: Use case diagram of PhishGuard

6.2 System Architecture

PhishGuard follows a client-server design: the browser extension performs interception, local feature extraction and caching, while the server performs domain lookups and model inference.

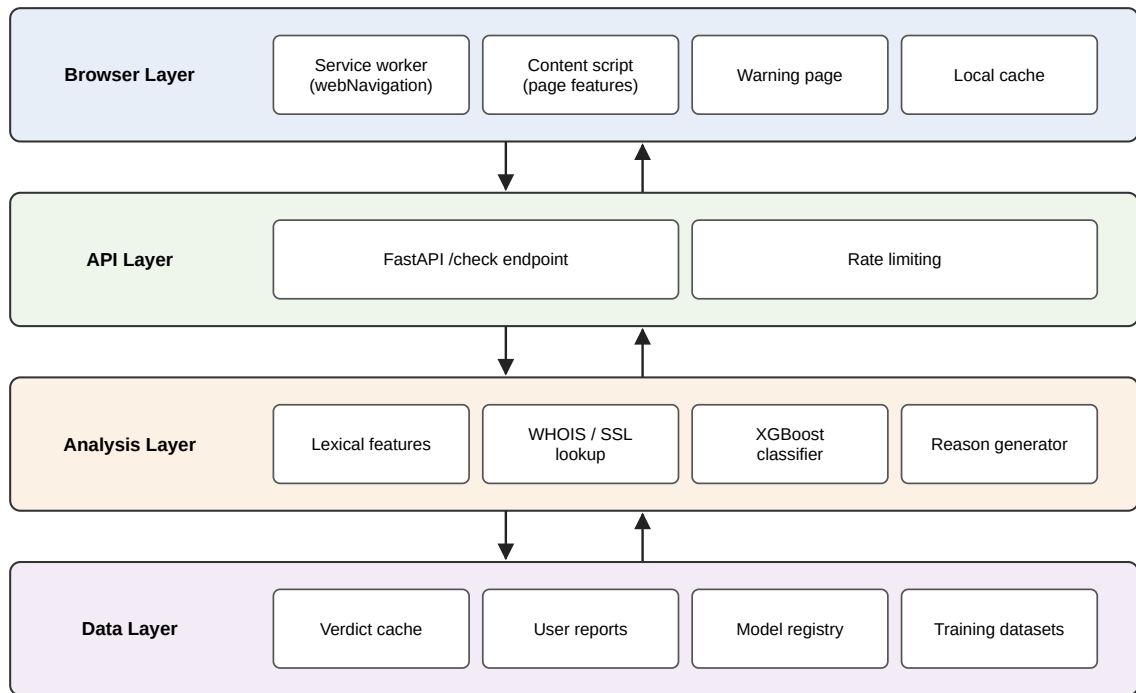


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

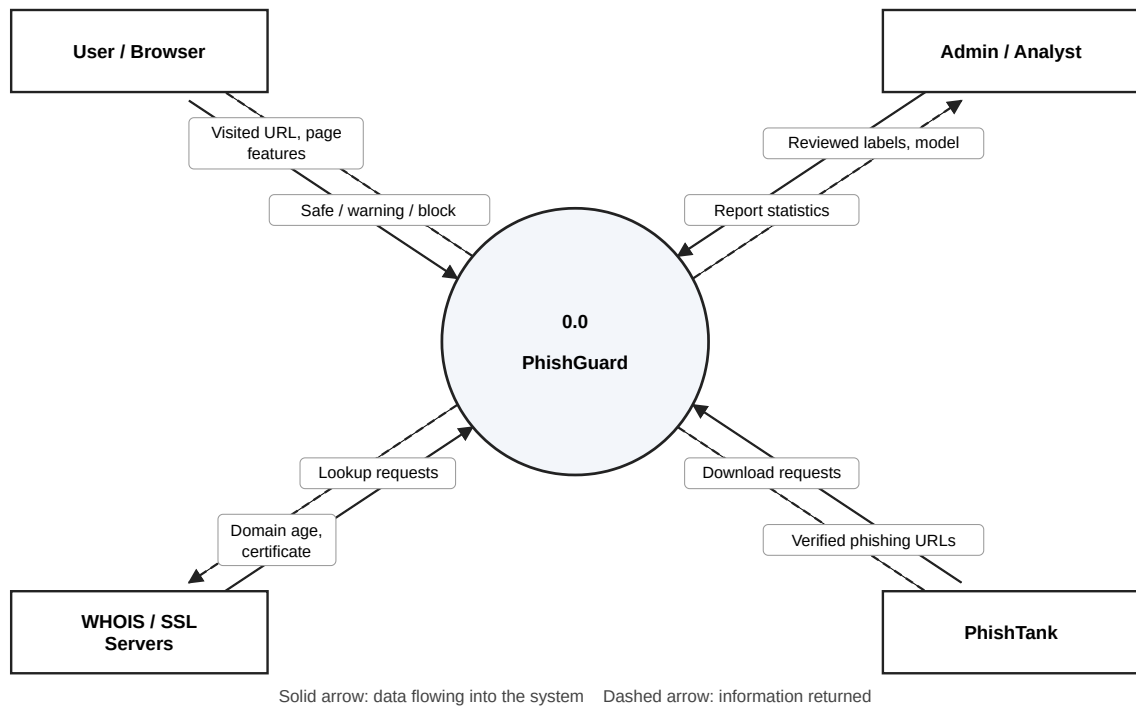


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

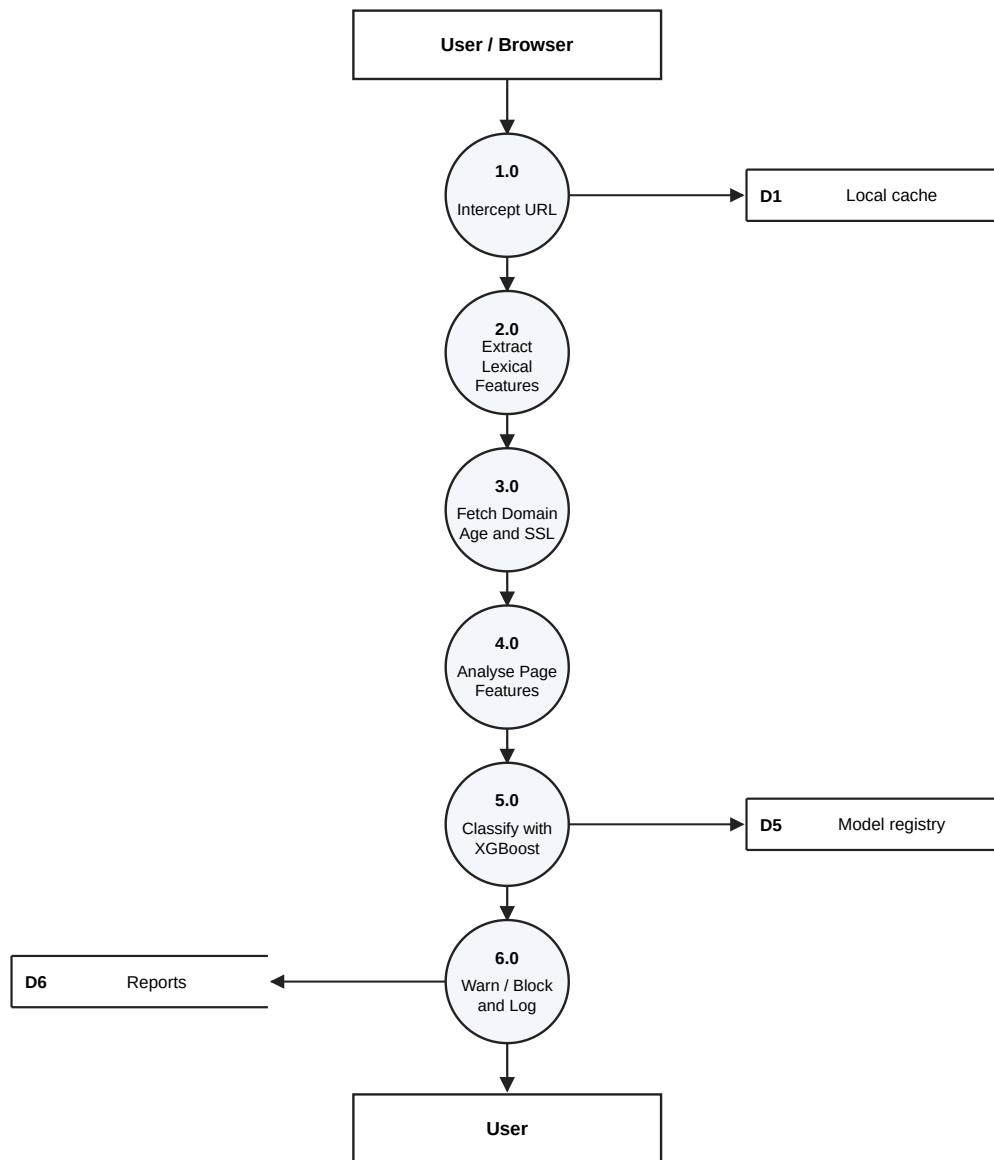


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

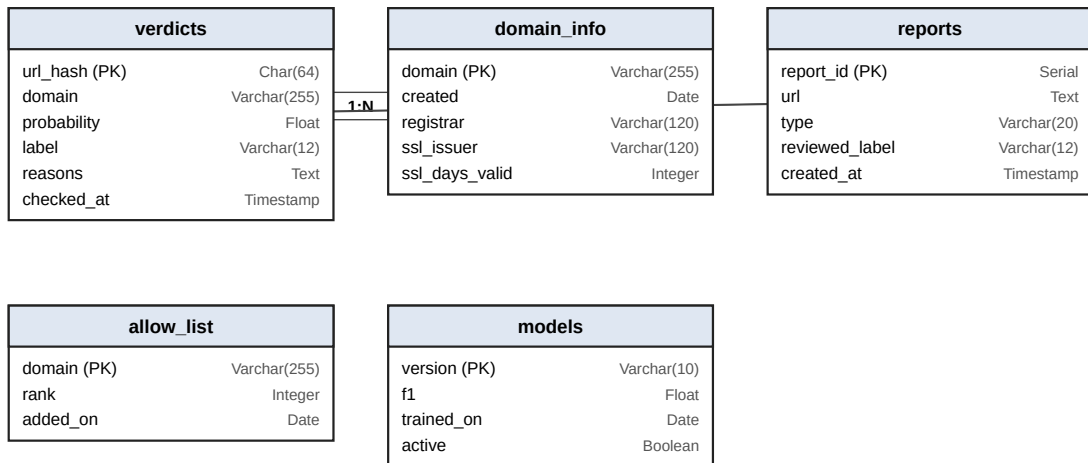


Fig 5: Entity relationship diagram

6.5 Table Design

VERDICTS

FIELD NAME	DATA TYPE	DESCRIPTION
url_hash	Char(64)	Primary key (SHA-256 of URL)
domain	Varchar(255)	Registered domain
probability	Float	Phishing probability
label	Varchar(12)	safe / suspicious / phishing
reasons	Text	JSON list of reasons
checked_at	Timestamp	Time

DOMAIN_INFO

FIELD NAME	DATA TYPE	DESCRIPTION
domain	Varchar(255)	Primary key
created	Date	Registration date
registrar	Varchar(120)	Registrar
ssl_issuer	Varchar(120)	Certificate issuer
ssl_days_valid	Integer	Validity period

REPORTS

FIELD NAME	DATA TYPE	DESCRIPTION
report_id	Serial	Primary key
url	Text	Reported URL
type	Varchar(20)	false_positive / missed_phish
reviewed_label	Varchar(12)	Analyst label
created_at	Timestamp	Time

ALLOW_LIST

FIELD NAME	DATA TYPE	DESCRIPTION
domain	Varchar(255)	Primary key
rank	Integer	Popularity rank
added_on	Date	Date

MODELS

FIELD NAME	DATA TYPE	DESCRIPTION
version	Varchar(10)	Primary key
f1	Float	F1 on test set
trained_on	Date	Training date
active	Boolean	In use

6.6 Algorithm Design

The detection algorithm runs on every top-level navigation.

1. **Intercept URL on navigation** – chrome.webNavigation.onBeforeNavigate gives the URL; allow-listed and cached URLs are resolved locally.
2. **Extract lexical features (length, @, hyphens, IP use)** – Also number of subdomains, digits ratio, Shannon entropy, suspicious words (login, verify, kyc, update, secure), shorteners and risky TLDs.

3. **Fetch domain age and SSL details** – python-whois gives creation date; the SSL certificate is fetched to get issuer and validity days.
4. **Analyse page features (forms, hidden iframes)** – Content script counts password inputs, forms with external action, hidden iframes, external resource ratio and brand-domain mismatch.
5. **Classify with Random Forest/XGBoost** – The model outputs phishing probability; 0.5-0.8 suspicious (warning), ≥ 0.8 phishing (block).
6. **Block page and show warning if risky** – The tab is redirected to the extension's warning page with reasons generated from the strongest risk features.
7. **Allow user report of false results** – Reports are sent to the server for analyst review.
8. **Update model periodically** – Weekly retraining with new PhishTank URLs and reviewed reports.

PSEUDO CODE

```
ON NAVIGATE(url):
  d <- REGISTERED_DOMAIN(url)
  IF d IN ALLOW_LIST OR CACHE.has(url) THEN RETURN cached verdict
  f <- LEXICAL(url)
  f <- f + SERVER.domain_features(d)           // age, ssl
  f <- f + CONTENT_SCRIPT.page_features()      // forms, iframes, links
  p <- XGB.predict_proba(f)
  IF p  $\geq$  0.8 THEN BLOCK(tab, REASONS(f))
  ELSE IF p  $\geq$  0.5 THEN WARN(tab, REASONS(f))
  CACHE.set(url, p, 24 h)
```

6.7 Modules

The modules are divided between the User (browser extension) and the Admin/Analyst (server side).

MODULES

USER

1. INSTALL AND BROWSE
2. VIEW WARNING
3. REPORT SITE

4. VIEW HISTORY

ADMIN / ANALYST

1. REVIEW REPORTS
2. RETRAIN AND PUBLISH MODEL
3. MANAGE ALLOW-LIST

USER

INSTALL AND BROWSE

The user installs the extension; every navigation is checked automatically. The toolbar icon shows green (safe), amber (suspicious) or red (phishing).

VIEW WARNING

For risky URLs, a full-page warning shows the risk score and reasons such as 'Domain created 2 days ago' or 'Password form sends data to another site'.

REPORT SITE

The user can report a missed phishing site or a wrongly blocked legitimate site.

VIEW HISTORY

The user sees recently blocked and warned sites.

ADMIN / ANALYST

REVIEW REPORTS

Analysts review user reports and confirm labels.

RETRAIN AND PUBLISH MODEL

Analysts retrain the model with new PhishTank data and reviewed reports and publish a new version.

MANAGE ALLOW-LIST

Maintain an allow-list of verified popular domains to reduce false positives.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

PhishGuard was implemented as a Chrome Manifest V3 extension with a background service worker and a content script, and a Python FastAPI server hosting the XGBoost model. Training used 25,000 phishing URLs from PhishTank (verified, online at collection time) and 25,000 legitimate URLs sampled from the Tranco top-sites list including internal pages.

7.2 Implementation

Page features for training were collected by a headless browser crawler (Playwright) at the time of URL collection, since phishing pages go offline quickly. The data was split by collection date to evaluate performance on newer phishing sites.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Legitimate site	https://www.onlinesbi.sbi	Safe (allow-list)	Pass
TC02	IP-based URL	http://103.x.x.x/kyc/update	Blocked with reasons	Pass
TC03	New look-alike domain	sbi-kyc-update-online.xyz (3 days old)	Blocked	Pass
TC04	HTTPS phishing	Valid DV certificate, fake login	Blocked - form posts to other domain	Pass
TC05	Shortened URL	bit.ly link to phishing page	Final URL checked and blocked	Pass
TC06	False positive report	Report wrongly blocked site	Report saved for review	Pass
TC07	Cache	Revisit same URL	Verdict from cache < 10 ms	Pass
TC08	Offline mode	Server unreachable	Local lexical model used	Pass

PERFORMANCE EVALUATION

Results on the time-split test set (newest 20% of URLs):

Model / features	Accuracy	Precision	Recall	F1
Logistic Regression (lexical)	89.7%	0.90	0.89	0.89
Random Forest (all features)	96.4%	0.97	0.96	0.96
XGBoost (all features, proposed)	97.6%	0.98	0.97	0.97
Google Safe Browsing at first visit*	-	1.00	0.41	0.58

7.4 Source Code

extension/manifest.json

```
{
  "manifest_version": 3,
```

```

    "name": "PhishGuard",
    "version": "1.2.0",
    "description": "Real-time phishing website detection",
    "permissions": ["webNavigation", "storage", "tabs", "scripting"],
    "host_permissions": ["<all_urls>"],
    "background": { "service_worker": "background.js" },
    "content_scripts": [{ "matches": ["<all_urls>"], "js": ["content.js"],
    "run_at": "document_idle" }],
    "action": { "default_popup": "popup.html", "default_icon": "icons/shield-
48.png" },
    "web_accessible_resources": [{ "resources": ["warning.html"], "matches": ["
<all_urls>"] }]
}

```

extension/background.js

```

const API = "https://api.phishguard.app/check";
const ALLOW = new Set(["google.com", "sbi", "onlinesbi.sbi", "hdfcbank.com",
"icicibank.com",
                                "amazon.in", "flipkart.com", "gov.in", "npci.org.in",
"wikipedia.org"]);

function lexical(u) {
    const url = new URL(u), host = url.hostname, s = u.toLowerCase();
    const entropy = [...new Set(s)].reduce((e, c) => {
        const p = s.split(c).length / s.length; return e - p * Math.log2(p); }, 0);
    return {
        url_len: u.length, host_len: host.length, dots: (host.match(/\./g) ||
[]).length,
        hyphens: (host.match(/-/g) || []).length, at: s.includes("@") ? 1 : 0,
        ip: /^d+\d+\.\d+\.\d+\.\d+$/i.test(host) ? 1 : 0, https: url.protocol ===
"https:" ? 1 : 0,
        digits: (s.match(/\d/g) || []).length / s.length, entropy,
        keywords: ["login", "verify", "kyc", "update", "secure", "account", "bank",
"reward", "otp"]
            .filter(k => s.includes(k)).length,
        tld_risky: /\.(xyz|top|tk|ml|ga|cf|gq|work|click|buzz)$/i.test(host) ? 1 : 0,
    };
}

const baseDomain = h => h.split(".").slice(-2).join(".");

chrome.webNavigation.onCommitted.addListener(async ({ tabId, url, frameId }) =>
{
    if (frameId !== 0 || !url.startsWith("http")) return;
    const host = new URL(url).hostname;
    if (ALLOW.has(baseDomain(host)) || host.endsWith(".gov.in")) return
setIcon(tabId, "safe");
    const cached = (await chrome.storage.local.get(url))[url];
    if (cached && Date.now() - cached.t < 86400000) return act(tabId, url,
cached.v);

    const [{ result: page }] = await chrome.scripting.executeScript({ target: {
tabId },
        func: () => window.__phishguardFeatures || null });
    const res = await fetch(API, { method: "POST", headers: { "Content-Type":
"application/json" },
                                body: JSON.stringify({ url, lexical:
lexical(url), page }) });
    const v = await res.json();
    await chrome.storage.local.set({ [url]: { v, t: Date.now() } });

```

```

    act(tabId, url, v);
  });

function act(tabId, url, v) {
  setIcon(tabId, v.label);
  if (v.label === "phishing" || v.label === "suspicious") {
    const q = new URLSearchParams({ url, p: v.probability, r:
JSON.stringify(v.reasons), l: v.label });
    chrome.tabs.update(tabId, { url: chrome.runtime.getURL("warning.html?" + q)
  });
  }
}

function setIcon(tabId, label) {
  chrome.action.setBadgeText({ tabId, text: label === "safe" ? "" : "!" });
  chrome.action.setBadgeBackgroundColor({ tabId, color: label === "phishing" ?
"#d6336c" : "#e8890c" });
}

```

extension/content.js

```

(() => {
  const host = location.hostname;
  const ext = el => { try { return new URL(el, location.href).hostname !== host;
} catch { return false; } };
  const forms = [...document.forms];
  const links = [...document.querySelectorAll("a[href]")].map(a => a.href);
  const brands = ["sbi", "hdfc", "icici", "paytm", "phonepe", "amazon",
"flipkart", "india post", "income tax"];
  const title = document.title.toLowerCase();
  window.__phishguardFeatures = {
    password_fields: document.querySelectorAll("input[type=password]").length,
    external_form_action: forms.filter(f => f.action && ext(f.action)).length,
    hidden_iframes: [...document.querySelectorAll("iframe")].filter(f =>
      f.width === "0" || f.height === "0" || getComputedStyle(f).display ===
"none").length,
    external_link_ratio: links.length ? links.filter(ext).length / links.length
: 0,
    brand_mismatch: brands.some(b => title.includes(b) &&
!host.includes(b.split(" ")[0])) ? 1 : 0,
    right_click_disabled: document.oncontextmenu ? 1 : 0,
  };
})();

```

server/api.py

```

import datetime as dt, socket, ssl, json, joblib, whois, tldextract, pandas as
pd
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
model = joblib.load("models/phishguard_xgb.pkl")
FEATS = list(model.get_booster().feature_names)
REASONS = {"domain_age_days": "Domain was registered only {v:.0f} days ago",
"ip": "Uses an IP address instead of a domain name",
          "external_form_action": "Login form sends your data to a different
website", "brand_mismatch": "Page claims to be a known brand but the domain does
not match",
          "keywords": "URL contains words like login/verify/KYC", "tld_risky":

```

```

"Uses a domain ending often used by scam sites",
    "hidden_iframes": "Page contains hidden frames", "ssl_days_valid":
"Short-term free SSL certificate"}

class Req(BaseModel):
    url: str; lexical: dict; page: dict | None = None

def domain_features(domain):
    age, days_valid = 3650, 365
    try:
        c = whois.whois(domain).creation_date
        c = c[0] if isinstance(c, list) else c
        age = (dt.datetime.now() - c).days
    except Exception:
        age = -1
    try:
        ctx = ssl.create_default_context()
        with ctx.wrap_socket(socket.create_connection((domain, 443), timeout=3),
server_hostname=domain) as s:
            cert = s.getpeercert()
            nb = dt.datetime.strptime(cert["notBefore"], "%b %d %H:%M:%S %Y %Z")
            na = dt.datetime.strptime(cert["notAfter"], "%b %d %H:%M:%S %Y %Z")
            days_valid = (na - nb).days
    except Exception:
        days_valid = 0
    return {"domain_age_days": age, "ssl_days_valid": days_valid}

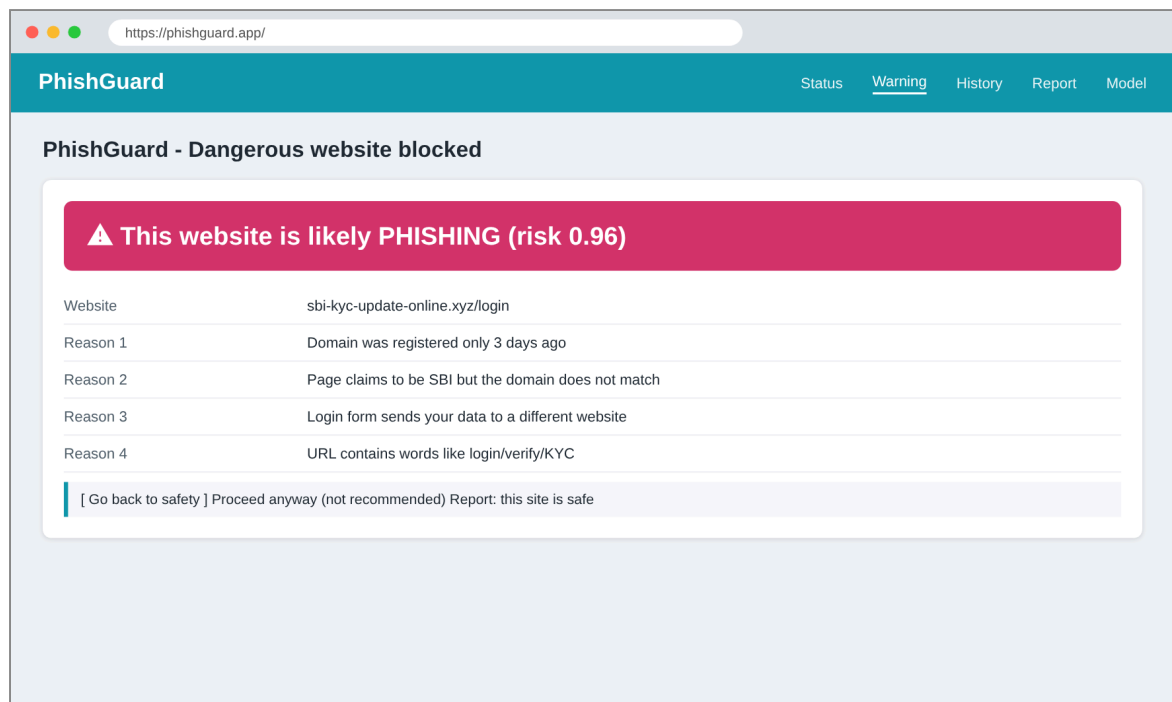
@app.post("/check")
def check(r: Req):
    d = tldextract.extract(r.url).registered_domain
    f = {**r.lexical, **domain_features(d), **(r.page or {})}
    X = pd.DataFrame([k: f.get(k, 0) for k in FEATS])
    p = float(model.predict_proba(X)[0, 1])
    label = "phishing" if p >= 0.8 else "suspicious" if p >= 0.5 else "safe"
    reasons = []
    if label != "safe":
        if 0 <= f["domain_age_days"] < 90:
            reasons.append(REASONS["domain_age_days"].format(v=f["domain_age_days"]))
        for k in ("ip", "external_form_action", "brand_mismatch", "tld_risky",
"hidden_iframes"):
            if f.get(k): reasons.append(REASONS[k].format(v=f[k]))
        if f.get("keywords", 0) >= 2: reasons.append(REASONS["keywords"])
    return {"probability": round(p, 3), "label": label, "reasons": reasons[:4]}

```

7.5 Screen Layout

SCREENSHOTS:

PHISHING WARNING PAGE

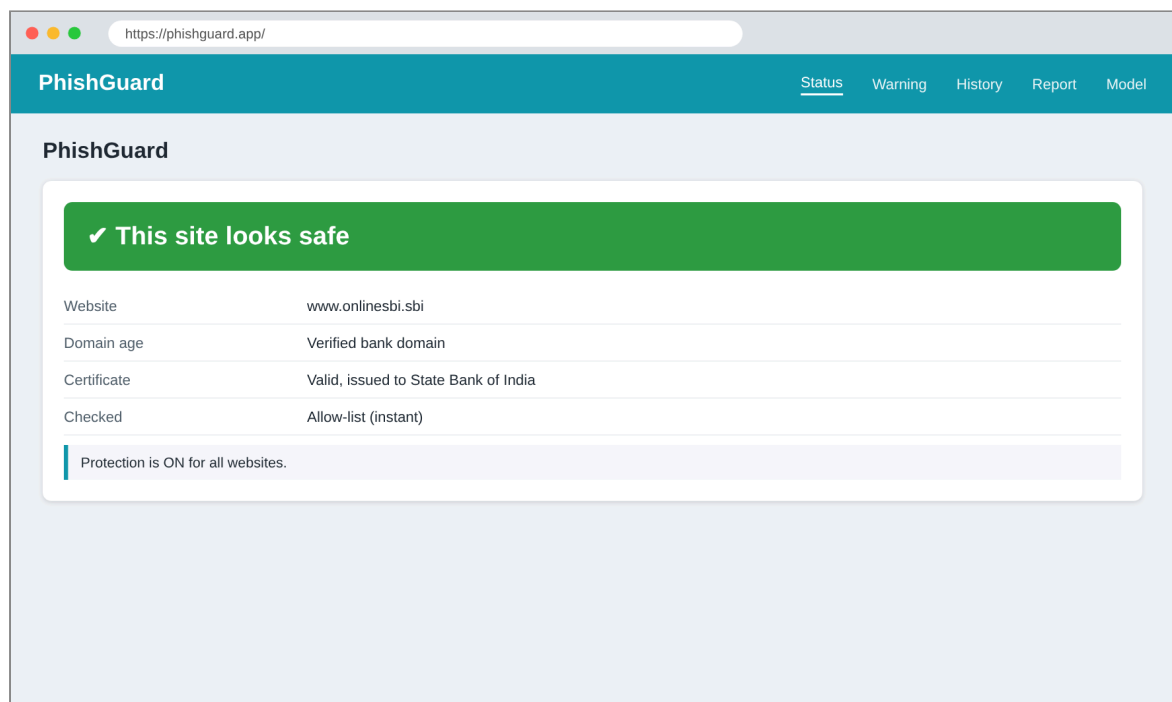


The screenshot shows the PhishGuard application interface. The browser address bar displays `https://phishguard.app/`. The application header includes the PhishGuard logo and navigation links: Status, Warning, History, Report, and Model. The main heading reads "PhishGuard - Dangerous website blocked". Below this, a red alert box states: "⚠ This website is likely PHISHING (risk 0.96)". A table lists the reasons for the warning:

Website	sbi-kyc-update-online.xyz/login
Reason 1	Domain was registered only 3 days ago
Reason 2	Page claims to be SBI but the domain does not match
Reason 3	Login form sends your data to a different website
Reason 4	URL contains words like login/verify/KYC

At the bottom of the table, a link is provided: "[Go back to safety] Proceed anyway (not recommended) Report: this site is safe".

EXTENSION POPUP - SAFE SITE



The screenshot shows the PhishGuard application interface. The browser address bar displays `https://phishguard.app/`. The application header includes the PhishGuard logo and navigation links: Status, Warning, History, Report, and Model. The main heading reads "PhishGuard". Below this, a green confirmation box states: "✓ This site looks safe". A table lists the verification details:

Website	www.onlinesbi.sbi
Domain age	Verified bank domain
Certificate	Valid, issued to State Bank of India
Checked	Allow-list (instant)

At the bottom of the table, a message is displayed: "Protection is ON for all websites."

BLOCKED SITES HISTORY

https://phishguard.app/

PhishGuard

StatusWarningHistoryReportModel

Recently warned / blocked

Time	Website	Risk	Action
14-10 19:21	sbi-kyc-update-online.xyz	0.96	Blocked
13-10 10:02	indiapost-redelivery.top	0.93	Blocked
12-10 21:45	free-recharge-offer.click	0.71	Warned - went back
11-10 08:30	amazOn-rewards.in	0.89	Blocked

ADMIN - MODEL AND REPORTS

https://phishguard.app/

PhishGuard

StatusWarningHistoryReportModel

Model v1.2 and user reports

Checks today
48,210

Blocked
312

Reports pending
27

Test F1
0.97

Blocked phishing pages per day

Day	Blocked phishing pages
Mon	260
Tue	270
Wed	260
Thu	340
Fri	280
Sat	250
Sun	220

Top impersonated brands

Brand	Blocked
Banks (KYC)	41%
India Post / courier	22%
E-commerce rewards	17%
Electricity bill	9%

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Phishing Website and Malicious URL Detection Browser Extension protects users in real time by analysing URL, domain and page content features with an XGBoost classifier, achieving an F1-score of 0.97 on newer phishing sites, including many not yet listed on blacklists at first visit. Warnings explain the danger in simple language, helping users learn to recognise scams.

The feedback loop and periodic retraining allow the model to adapt to changing attacker techniques. The extension is lightweight and privacy-conscious, making it suitable for students, elderly users and organisations.

8.2 Future Enhancement

- Visual similarity detection of login pages using screenshots and CNNs.
- SMS and WhatsApp link scanning through a mobile app.
- Federated learning from user reports while preserving privacy.
- Support for Firefox and Edge.
- Integration with CERT-In and [cybercrime.gov.in](https://www.cybercrime.gov.in) reporting.

CHAPTER IX

BIBLIOGRAPHY

1. Zhang, Y., Hong, J. I., and Cranor, L. F., "CANTINA: A Content-Based Approach to Detecting Phishing Web Sites", Proc. WWW, pp. 639-648, 2007.
2. Ma, J., Saul, L. K., Savage, S., and Voelker, G. M., "Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs", Proc. ACM SIGKDD, 2009.
3. Mohammad, R. M., Thabtah, F., and McCluskey, L., "An Assessment of Features Related to Phishing Websites using an Automated Technique", Proc. ICITST, 2012.
4. Sahingoz, O. K., Buber, E., Demir, O., and Diri, B., "Machine learning based phishing detection from URLs", Expert Systems with Applications, Vol. 117, pp. 345-357, 2019.
5. Anti-Phishing Working Group, "Phishing Activity Trends Report", APWG, quarterly reports.
6. Chen, T., and Guestrin, C., "XGBoost: A Scalable Tree Boosting System", Proc. ACM SIGKDD, 2016.

WEBSITE REFERENCE

1. <https://phishtank.org>
2. <https://developer.chrome.com/docs/extensions/mv3>
3. <https://tranco-list.eu>
4. <https://cybercrime.gov.in>